

数论选讲

前置知识

同余，整除，质数，gcd/lcm，分解质因数



前言

如果把实数放在模意义下进行运算，好处是可以充分利用同余不改变加减乘除性质，使得实数运算可以变为整数运算。

离散数学就是建立在模意义下的一个数学方法的类别。

定义

我们说一个形如 $Ax \equiv B \pmod{m}$ 的式子为同余方程。



裴蜀定理

定理

设 a, b 是不全为零的整数, 则存在整数 x, y , 使得 $ax + by = p$, 那么充要条件是 $\gcd(a, b) | p$.



裴蜀定理

定理

设 a, b 是不全为零的整数, 则存在整数 x, y , 使得 $ax + by = p$, 那么充要条件是 $\gcd(a, b) | p$.

扩展

设 n 元不定方程为 $a_1x_1 + a_2x_2 + \dots + a_nx_n = p$, 其有解的充要条件是 $\gcd(a_1, a_2, \dots, a_n) | p$

即若 $\gcd(a, b) \mid p$, 则 $ax + by = p$ 有解。



充分性证明

即若 $\gcd(a, b) \mid p$, 则 $ax + by = p$ 有解。

首先要证 $ax + by = \gcd(a, b)$ 有解。



充分性证明

即若 $\gcd(a, b) | p$, 则 $ax + by = p$ 有解。

首先要证 $ax + by = \gcd(a, b)$ 有解。

设 $ax_1 + by_1 = \gcd(a, b)$

于是 $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$

由欧几里得定理可知: $\gcd(a, b) = \gcd(b, a \bmod b)$

所以

$$ax_1 + by_1 = bx_2 + (a \bmod b)y_2$$

$$ax_1 + by_1 = bx_2 + (a - (\lfloor \frac{a}{b} \rfloor \times b))y_2$$

$$ax_1 + by_1 = ay_2 + bx_2 - \lfloor \frac{a}{b} \rfloor \times by_2 = ay_2 + b(x_2 - \lfloor \frac{a}{b} \rfloor y_2)$$

所以 $x_1 = y_2, y_1 = x_2 - \lfloor \frac{a}{b} \rfloor y_2$

将 x_2, y_2 不断代入递归求解直至 $\gcd$ 为 0 递归 $x = 1, y = 0$ 回去求解。



充分性证明

即若 $\gcd(a, b) | p$, 则 $ax + by = p$ 有解。

首先要证 $ax + by = \gcd(a, b)$ 有解。

设 $ax_1 + by_1 = \gcd(a, b)$

于是 $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$

由欧几里得定理可知: $\gcd(a, b) = \gcd(b, a \bmod b)$

所以

$$ax_1 + by_1 = bx_2 + (a \bmod b)y_2$$

$$ax_1 + by_1 = bx_2 + (a - (\lfloor \frac{a}{b} \rfloor \times b))y_2$$

$$ax_1 + by_1 = ay_2 + bx_2 - \lfloor \frac{a}{b} \rfloor \times by_2 = ay_2 + b(x_2 - \lfloor \frac{a}{b} \rfloor y_2)$$

所以 $x_1 = y_2, y_1 = x_2 - \lfloor \frac{a}{b} \rfloor y_2$

将 x_2, y_2 不断代入递归求解直至 $\gcd$ 为 0 递归 $x = 1, y = 0$ 回去求解。

可以发现通过上述过程一定能找到一组解 (这就是扩展欧几里得算法)



充分性证明

即若 $\gcd(a, b) | p$, 则 $ax + by = p$ 有解。

首先要证 $ax + by = \gcd(a, b)$ 有解。

设 $ax_1 + by_1 = \gcd(a, b)$

于是 $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$

由欧几里得定理可知: $\gcd(a, b) = \gcd(b, a \bmod b)$

所以

$$ax_1 + by_1 = bx_2 + (a \bmod b)y_2$$

$$ax_1 + by_1 = bx_2 + (a - (\lfloor \frac{a}{b} \rfloor \times b))y_2$$

$$ax_1 + by_1 = ay_2 + bx_2 - \lfloor \frac{a}{b} \rfloor \times by_2 = ay_2 + b(x_2 - \lfloor \frac{a}{b} \rfloor y_2)$$

所以 $x_1 = y_2, y_1 = x_2 - \lfloor \frac{a}{b} \rfloor y_2$

将 x_2, y_2 不断代入递归求解直至 $\gcd$ 为 0 递归 $x = 1, y = 0$ 回去求解。

可以发现通过上述过程一定能找到一组解 (这就是扩展欧几里得算法)

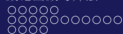
而找到一组解后, 因为 $\gcd(a, b) | p$, 故把解 x_1, y_1 同时扩展 $p / \gcd(a, b)$ 倍即可得到一个解。



必要性证明

这个比较简单，即要证若 $ax + by = m$ 有解，则 $\gcd(a, b) \mid m$ 。
因为 $\gcd(a, b) \mid a$, $\gcd(a, b) \mid b$, 故 $\gcd(a, b) \mid ax + by$

例题 CF510D Fox And Jumping



例题 CF510D Fox And Jumping

转化一下题意，等价于选出若干个元素 $l_{p_1}, l_{p_2}, \dots, l_{p_k}$ 使得方程 $l_{p_1}x_1 + l_{p_2}x_2 + \dots + l_{p_k}x_k = m$ 能取遍所有自然数。



例题 CF510D Fox And Jumping

转化一下题意，等价于选出若干个元素 $l_{p_1}, l_{p_2}, \dots, l_{p_k}$ 使得方程

$l_{p_1}x_1 + l_{p_2}x_2 + \dots + l_{p_k}x_k = m$ 能取遍所有自然数。

话句话说 $\gcd(l_{p_1}, l_{p_2}, \dots, l_{p_k}) = 1$ ，于是考虑 dp ，设 $f_{i,j}$ 表示考虑了前 i 个数，目前的 $\gcd = j$ 的最小代价。

例题 CF510D Fox And Jumping

转化一下题意，等价于选出若干个元素 $l_{p_1}, l_{p_2}, \dots, l_{p_k}$ 使得方程

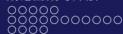
$l_{p_1}x_1 + l_{p_2}x_2 + \dots + l_{p_k}x_k = m$ 能取遍所有自然数。

话句话说 $\gcd(l_{p_1}, l_{p_2}, \dots, l_{p_k}) = 1$ ，于是考虑 dp ，设 $f_{i,j}$ 表示考虑了前 i 个数，目前的 $\gcd = j$ 的最小代价。

dp 数组只有 l_i 因子处有值，故时间复杂度为 $O(n \sum d(l_i))$

与同余方程的关系

对于一个同余方程 $Ax \equiv B \pmod{m}$ 来说，它等价于 $Ax + my = B$ 。



与同余方程的关系

对于一个同余方程 $Ax \equiv B \pmod{m}$ 来说，它等价于 $Ax + my = B$ 。
所以一些同余方程可以转化为二元不定方程，利用裴蜀定理和扩展欧几里得算法解决。

逆元

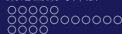
逆元解决的是模意义下的除法问题，即 x 的逆元 a 满足

$$ax \equiv 1 \pmod{m}$$

也会记作 x^{-1}

如何求逆元

我们把上面定义逆元的同余方程写成二元不定方程 $ax + my = 1$ ，于是可以直接使用扩展欧几里得算法求解。



如何求逆元

我们把上面定义逆元的同余方程写成二元不定方程 $ax + my = 1$ ，于是可以直接使用扩展欧几里得算法求解。

根据裴蜀定理，有解的充要条件是 $\gcd(a, m) = 1$



如何求逆元 2

费马小定理

若 m 是素数且 $\gcd(a, m) = 1$, 那么 $a^{m-1} \equiv 1 \pmod{m}$

于是逆元就是 $a^{m-2} \bmod m$ 。

如何求逆元 2

费马小定理

若 m 是素数且 $\gcd(a, m) = 1$, 那么 $a^{m-1} \equiv 1 \pmod{m}$

于是逆元就是 $a^{m-2} \bmod m$ 。

这种方法更为常用, 因为一个快速幂就可以解决。但是限制相比前一种算法更严格要求了 m 为质数, 某些特殊情况下可能失效。

例题洛谷 P5431 乘法逆元 2



(扩展) 中国剩余定理定义

有 n 个同余方程：

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \dots \\ x \equiv a_n \pmod{m_n} \end{cases}$$

求满足上面所有方程的解的方程。

(扩展) 中国剩余定理过程

扩展中国剩余定理的主要思想是每次将两个同余方程合并为一个方程，这样合并若干次就能得到答案。

(扩展) 中国剩余定理过程

扩展中国剩余定理的主要思想是每次将两个同余方程合并为一个方程，这样合并若干次就能得到答案。

假设我们要合并的两个方程是 $x \equiv a_1 \pmod{m_1}$ 和 $x \equiv a_2 \pmod{m_2}$ ，那么我们把它们写成不定方程，即 $x = m_1p + a_1 = m_2q + a_2$ 。

(扩展) 中国剩余定理过程

扩展中国剩余定理的主要思想是每次将两个同余方程合并为一个方程，这样合并若干次就能得到答案。

假设我们要合并的两个方程是 $x \equiv a_1 \pmod{m_1}$ 和 $x \equiv a_2 \pmod{m_2}$ ，那么我们把它们写成不定方程，即 $x = m_1p + a_1 = m_2q + a_2$ 。

移项后得到 $m_1p - m_2q = a_2 - a_1$ ，这个不定方程有解的充要条件是 $\gcd(m_1, m_2) | a_2 - a_1$ ，若有解那么可以通过扩展欧几里得算法得到一组解 p, q 。

(扩展) 中国剩余定理过程

扩展中国剩余定理的主要思想是每次将两个同余方程合并为一个方程，这样合并若干次就能得到答案。

假设我们要合并的两个方程是 $x \equiv a_1 \pmod{m_1}$ 和 $x \equiv a_2 \pmod{m_2}$ ，那么我们把它们写成不定方程，即 $x = m_1p + a_1 = m_2q + a_2$ 。

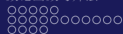
移项后得到 $m_1p - m_2q = a_2 - a_1$ ，这个不定方程有解的充要条件是 $\gcd(m_1, m_2) | a_2 - a_1$ ，若有解那么可以通过扩展欧几里得算法得到一组解 p, q 。于是我们能算出 x 的一个解 $m_1p + a_1$ ，有 $x \equiv m_1p + a_1 \pmod{\text{lcm}(m_1, m_2)}$

例题洛谷 P3868 猜数字



例题洛谷 P3868 猜数字

$b_i | (n - a_i)$ 等价于 $n - a_i \equiv 0 \pmod{b_i}$, 移项得到 $n \equiv a_i \pmod{b_i}$



例题洛谷 P3868 猜数字

$b_i | (n - a_i)$ 等价于 $n - a_i \equiv 0 \pmod{b_i}$, 移项得到 $n \equiv a_i \pmod{b_i}$
相当于求 n 个同余方程的公共解, 使用 (扩展) 中国剩余定理即可。

例题洛谷 P4774 屠龙勇士

例题洛谷 P4774 屠龙勇士

这题的同余方程相比常规中国剩余定理的方程多了系数。同时因为其与模数不互质，所以没法求逆元直接转化为常规中国剩余定理求解。



例题洛谷 P4774 屠龙勇士

这题的同余方程相比常规中国剩余定理的方程多了系数。同时因为其与模数不互质，所以没法求逆元直接转化为常规中国剩余定理求解。

于是我们想要把 $bx \equiv a \pmod{m}$ 的式子化成我们熟知的 $x \equiv a' \pmod{m}'$ 。



例题洛谷 P4774 屠龙勇士

这题的同余方程相比常规中国剩余定理的方程多了系数。同时因为其与模数不互质，所以没法求逆元直接转化为常规中国剩余定理求解。

于是我们想要把 $bx \equiv a \pmod{m}$ 的式子化成我们熟知的 $x \equiv a' \pmod{m}'$ 。先化成二元不定方程 $bx + my = a$ ，可以先保证 $\gcd(b, m) | a$ ，接下来只需要类似扩欧一样找到一组通解 x', y' ，于是有 $x = x' + k \frac{m}{\gcd(b, m)}$ ，化成同余方程就是 $x \equiv x' \pmod{\frac{m}{\gcd(b, m)}}$

例题洛谷 P4774 屠龙勇士

这题的同余方程相比常规中国剩余定理的方程多了系数。同时因为其与模数不互质，所以没法求逆元直接转化为常规中国剩余定理求解。

于是我们想要把 $bx \equiv a \pmod{m}$ 的式子化成我们熟知的 $x \equiv a' \pmod{m}'$ 。
先化成二元不定方程 $bx + my = a$ ，可以先保证 $\gcd(b, m) | a$ ，接下来只需要类似扩欧一样找到一组通解 x', y' ，于是有 $x = x' + k \frac{m}{\gcd(b, m)}$ ，化成同余方程就是 $x \equiv x' \pmod{\frac{m}{\gcd(b, m)}}$

接下来再跑普通的中国剩余定理就好了。



欧拉定理

定理

若 $a \perp m$, 那么 $a^{\varphi(m)} \equiv 1 \pmod{m}$



欧拉定理

定理

若 $a \perp m$, 那么 $a^{\varphi(m)} \equiv 1 \pmod{m}$

主要用途是对指数取模。

原根

阶

满足 $a^n \equiv 1 \pmod{m}$ 的最小正整数 n 称为 a 模 m 下的阶, 记作 $\text{ord}_m(a)$

原根

阶

满足 $a^n \equiv 1 \pmod{m}$ 的最小正整数 n 称为 a 模 m 下的阶, 记作 $\text{ord}_m(a)$

阶有一个比较好的性质, 即 $a, a^2, \dots, a^{\text{ord}_m(a)}$ 两两模 m 不同余。



原根

阶

满足 $a^n \equiv 1 \pmod{m}$ 的最小正整数 n 称为 a 模 m 下的阶, 记作 $\text{ord}_m(a)$

阶有一个比较好的性质, 即 $a, a^2, \dots, a^{\text{ord}_m(a)}$ 两两模 m 不同余。

可以把阶理解为一个循环节, 相当于 $a^0 \sim a^{\text{ord}_m(a)}$ 构成了一个循环。



原根

阶

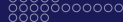
满足 $a^n \equiv 1 \pmod{m}$ 的最小正整数 n 称为 a 模 m 下的阶，记作 $\text{ord}_m(a)$

阶有一个比较好的性质，即 $a, a^2, \dots, a^{\text{ord}_m(a)}$ 两两模 m 不同余。

可以把阶理解为一个循环节，相当于 $a^0 \sim a^{\text{ord}_m(a)}$ 构成了一个循环。

原根

若 $\gcd(g, m) = 1$ 并且 $\text{ord}_m(g) = \varphi(m)$ ，那么 g 就是 m 的原根。



原根

阶

满足 $a^n \equiv 1 \pmod{m}$ 的最小正整数 n 称为 a 模 m 下的阶，记作 $\text{ord}_m(a)$

阶有一个比较好的性质，即 $a, a^2, \dots, a^{\text{ord}_m(a)}$ 两两模 m 不同余。

可以把阶理解为一个循环节，相当于 $a^0 \sim a^{\text{ord}_m(a)}$ 构成了一个循环。

原根

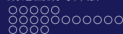
若 $\gcd(g, m) = 1$ 并且 $\text{ord}_m(g) = \varphi(m)$ ，那么 g 就是 m 的原根。

原根存在定理：模数 m 存在原根的充要条件是， m 为 $1, 2, 4$ 或存在奇质数 p 和自然数 α ，使得 $m = p^\alpha$ 或 $m = 2p^\alpha$

原根的数量：若 m 存在原根，那么它原根的数量为 $\varphi(\varphi(m))$ 。

最小原根大小：若 m 存在原根，那么它最小的原根是 $O(m^{0.25})$ 级别的。

那么我们如何求 m 的原根呢。



原根

那么我们如何求 m 的原根呢。

由前面的结论可以得知，最小的原根是非常小的，所以我们完全可以枚举一个数 g 看它是否是 m 的原根，于是问题转化为了判断原根。



原根

那么我们如何求 m 的原根呢。

由前面的结论可以得知，最小的原根是非常小的，所以我们完全可以枚举一个数 g 看它是否是 m 的原根，于是问题转化为了判断原根。

首先根据欧拉定理，有 $g^{\varphi(m)} \equiv 1 \pmod{m}$ ，因为存在循环节，所以 $\text{ord}_m(g) | \varphi(m)$ 。



原根

那么我们如何求 m 的原根呢。

由前面的结论可以得知，最小的原根是非常小的，所以我们完全可以枚举一个数 g 看它是否是 m 的原根，于是问题转化为了判断原根。

首先根据欧拉定理，有 $g^{\varphi(m)} \equiv 1 \pmod{m}$ ，因为存在循环节，所以 $\text{ord}_m(g) | \varphi(m)$ 。

所以枚举 $\varphi(m)$ 所有真因子，判断是否都不是阶就好了。判断一次时间复杂度 $O(\sqrt{m})$



原根

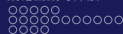
那么我们如何求 m 的原根呢。

由前面的结论可以得知，最小的原根是非常小的，所以我们完全可以枚举一个数 g 看它是否是 m 的原根，于是问题转化为了判断原根。

首先根据欧拉定理，有 $g^{\varphi(m)} \equiv 1 \pmod{m}$ ，因为存在循环节，所以 $\text{ord}_m(g) | \varphi(m)$ 。

所以枚举 $\varphi(m)$ 所有真因子，判断是否都不是阶就好了。判断一次时间复杂度 $O(\sqrt{m})$

总时间复杂度约为 $O(m^{0.75})$ ，实际远远达不到。



质数的原根

可以发现，如果一个质数 p 有原根 g ，由于 $\varphi(p) = p - 1$ ，又因为 $g, g^2, \dots, g^{p-1}$ 两两不同余，所以相当于 $1 \sim p - 1$ 在这之中全部出现了一次。



质数的原根

可以发现, 如果一个质数 p 有原根 g , 由于 $\varphi(p) = p - 1$, 又因为 $g, g^2, \dots, g^{p-1}$ 两两不同余, 所以相当于 $1 \sim p - 1$ 在这之中全部出现了一次。换句话说, 如果我们想要算 $ab \bmod m$, 我们可以令 $a = g^x, b = g^y$, 那么就是 $g^{x+y} \bmod m$, 可以发现我们很好地将乘法转化为了加法。这与实数中的对数函数十分类似。

BSGS 算法

大步小步算法，给定 a, b, m ，求解以下方程：

$$a^x \equiv b \pmod{m}$$

满足 $a \perp m$ 。

BSGS 算法

我们令 $B = \sqrt{m}$, 那么 $a^x = a^{x \bmod B + B \lfloor \frac{x}{B} \rfloor} = a^{x \bmod B} \times a^{B \lfloor \frac{x}{B} \rfloor}$

BSGS 算法

我们令 $B = \sqrt{m}$, 那么 $a^x = a^{x \bmod B + B \lfloor \frac{x}{B} \rfloor} = a^{x \bmod B} \times a^{B \lfloor \frac{x}{B} \rfloor}$

因为我们有限制 $x < \text{ord}_m(a)$, 所以 $\lfloor \frac{x}{B} \rfloor$ 的上界是 $O(m/B)$ 级别的。

BSGS 算法

我们令 $B = \sqrt{m}$, 那么 $a^x = a^{x \bmod B + B \lfloor \frac{x}{B} \rfloor} = a^{x \bmod B} \times a^{B \lfloor \frac{x}{B} \rfloor}$
 因为我们有限制 $x < \text{ord}_m(a)$, 所以 $\lfloor \frac{x}{B} \rfloor$ 的上界是 $O(m/B)$ 级别的。
 枚举 $\lfloor \frac{x}{B} \rfloor$ 于是有 $a^{x \bmod B} \equiv \frac{b}{a^{B \lfloor \frac{x}{B} \rfloor}} \pmod{m}$ (因为互质所以有逆元)。



BSGS 算法

我们令 $B = \sqrt{m}$, 那么 $a^x = a^{x \bmod B + B \lfloor \frac{x}{B} \rfloor} = a^{x \bmod B} \times a^{B \lfloor \frac{x}{B} \rfloor}$

因为我们有限制 $x < \text{ord}_m(a)$, 所以 $\lfloor \frac{x}{B} \rfloor$ 的上界是 $O(m/B)$ 级别的。

枚举 $\lfloor \frac{x}{B} \rfloor$ 于是有 $a^{x \bmod B} \equiv \frac{b}{a^{B \lfloor \frac{x}{B} \rfloor}} \pmod{m}$ (因为互质所以有逆元)。

又因为 $x \bmod B$ 的取值也只有 $O(B)$ 个, 所以我们预处理所有取值, 把它们放到一个 map 里, 然后枚举 $\lfloor \frac{x}{B} \rfloor$ 查表即可。

BSGS 算法

我们令 $B = \sqrt{m}$, 那么 $a^x = a^{x \bmod B + B \lfloor \frac{x}{B} \rfloor} = a^{x \bmod B} \times a^{B \lfloor \frac{x}{B} \rfloor}$

因为我们有限制 $x < \text{ord}_m(a)$, 所以 $\lfloor \frac{x}{B} \rfloor$ 的上界是 $O(m/B)$ 级别的。

枚举 $\lfloor \frac{x}{B} \rfloor$ 于是有 $a^{x \bmod B} \equiv \frac{b}{a^{B \lfloor \frac{x}{B} \rfloor}} \pmod{m}$ (因为互质所以有逆元)。

又因为 $x \bmod B$ 的取值也只有 $O(B)$ 个, 所以我们预处理所有取值, 把它们放到一个 map 里, 然后枚举 $\lfloor \frac{x}{B} \rfloor$ 查表即可。

时间复杂度 $O(\sqrt{m} \log m)$



BSGS 的变式

给定 a, b, m , 求解以下方程:

$$x^a \equiv b \pmod{m}$$

满足 m 是质数。

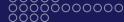
BSGS 的变式

给定 a, b, m , 求解以下方程:

$$x^a \equiv b \pmod{m}$$

满足 m 是质数。

找到 m 的原根 g , 那么令 $x = g^{x'}$, 方程就变为 $(g^{x'})^a \equiv b \pmod{m}$



BSGS 的变式

给定 a, b, m , 求解以下方程:

$$x^a \equiv b \pmod{m}$$

满足 m 是质数。

找到 m 的原根 g , 那么令 $x = g^{x'}$, 方程就变为 $(g^{x'})^a \equiv b \pmod{m}$

换一下变成 $(g^a)^{x'} \equiv b \pmod{m}$, 变成了常规 BSGS 能解决的问题。

例题洛谷 P3306 随机数生成器

例题洛谷 P3306 随机数生成器

题目给出的递推式其实就是 a 进制数，第一位是 x_1 ，其余每位都是 b 。



例题洛谷 P3306 随机数生成器

题目给出的递推式其实就是 a 进制数，第一位是 x_1 ，其余每位都是 b 。
令 $x_i = t$ ，那么根据公式可以写出 $t \equiv a^{i-1}x_1 + b \sum_{j=0}^{i-2} a^j \pmod{p}$



例题洛谷 P3306 随机数生成器

题目给出的递推式其实就是 a 进制数，第一位是 x_1 ，其余每位都是 b 。

令 $x_i = t$ ，那么根据公式可以写出 $t \equiv a^{i-1}x_1 + b \sum_{j=0}^{i-2} a^j \pmod{p}$

后面那一坨是等比数列求和，可以写成 $t \equiv a^{i-1}x_1 + b \frac{1-a^{i-1}}{1-a} \pmod{p}$



例题洛谷 P3306 随机数生成器

题目给出的递推式其实就是 a 进制数，第一位是 x_1 ，其余每位都是 b 。

令 $x_i = t$ ，那么根据公式可以写出 $t \equiv a^{i-1}x_1 + b \sum_{j=0}^{i-2} a^j \pmod{p}$

后面那一坨是等比数列求和，可以写成 $t \equiv a^{i-1}x_1 + b \frac{1-a^{i-1}}{1-a} \pmod{p}$

拆开得到 $t \equiv a^{i-1}x_1 + \frac{b}{1-a} - a^{i-1} \cdot \frac{b}{1-a} \pmod{p}$



例题洛谷 P3306 随机数生成器

题目给出的递推式其实就是 a 进制数，第一位是 x_1 ，其余每位都是 b 。

令 $x_i = t$ ，那么根据公式可以写出 $t \equiv a^{i-1}x_1 + b \sum_{j=0}^{i-2} a^j \pmod{p}$

后面那一坨是等比数列求和，可以写成 $t \equiv a^{i-1}x_1 + b \frac{1-a^{i-1}}{1-a} \pmod{p}$

拆开得到 $t \equiv a^{i-1}x_1 + \frac{b}{1-a} - a^{i-1} \cdot \frac{b}{1-a} \pmod{p}$

发现其中只有 a^{i-1} 与答案有关，其余都是常数，于是把 a^{i-1} 移到左边，其余移到右边得到

$$t \equiv a^{i-1} \left(x_1 - \frac{b}{1-a} \right) + \frac{b}{1-a} \pmod{p}$$

$$a^{i-1} \equiv \frac{t - \frac{b}{1-a}}{x_1 - \frac{b}{1-a}}$$



例题洛谷 P3306 随机数生成器

题目给出的递推式其实就是 a 进制数，第一位是 x_1 ，其余每位都是 b 。

令 $x_i = t$ ，那么根据公式可以写出 $t \equiv a^{i-1}x_1 + b \sum_{j=0}^{i-2} a^j \pmod{p}$

后面那一坨是等比数列求和，可以写成 $t \equiv a^{i-1}x_1 + b \frac{1-a^{i-1}}{1-a} \pmod{p}$

拆开得到 $t \equiv a^{i-1}x_1 + \frac{b}{1-a} - a^{i-1} \cdot \frac{b}{1-a} \pmod{p}$

发现其中只有 a^{i-1} 与答案有关，其余都是常数，于是把 a^{i-1} 移到左边，其余移到右边得到

$$t \equiv a^{i-1} \left(x_1 - \frac{b}{1-a} \right) + \frac{b}{1-a} \pmod{p}$$

$$a^{i-1} \equiv \frac{t - \frac{b}{1-a}}{x_1 - \frac{b}{1-a}}$$

是我们熟悉的形式，可以用 bsgs 解决。

例题 CF1106F Lunar New Year and...

例题 CF1106F Lunar New Year and...

题目给出的是乘法，很不好看。众所周知 998244353 由原根 $g = 3$ ，故利用原根将乘法转为加法，即令 $f_i = g^{F_i}$ ，那么 $g^{F_i} = g^{\sum_j b_j F_{i-j}}$ ，即 $F_i = \sum_j b_j F_{i-j}$



例题 CF1106F Lunar New Year and...

题目给出的是乘法，很不好看。众所周知 998244353 由原根 $g = 3$ ，故利用原根将乘法转为加法，即令 $f_i = g^{F_i}$ ，那么 $g^{F_i} = g^{\sum_j b_j F_{i-j}}$ ，即 $F_i = \sum_j b_j F_{i-j}$ 这是一个线性递推的形式，可以用矩阵快速幂优化。又因为 $F_1 = F_2 = \cdots = F_{k-1} = 0$ ，那么令 $F_k = 1$ ，这样算出来的 F_n 就是一个单位 F_k 贡献的次数，令其位 A ，而 $F_k = x$ 即有 x 个单位就是 $Ax \equiv F_n \pmod{998244352}$ 。



例题 CF1106F Lunar New Year and...

题目给出的是乘法，很不好看。众所周知 998244353 由原根 $g = 3$ ，故利用原根将乘法转为加法，即令 $f_i = g^{F_i}$ ，那么 $g^{F_i} = g^{\sum_j b_j F_{i-j}}$ ，即 $F_i = \sum_j b_j F_{i-j}$ 这是一个线性递推的形式，可以用矩阵快速幂优化。又因为 $F_1 = F_2 = \dots = F_{k-1} = 0$ ，那么令 $F_k = 1$ ，这样算出来的 F_n 就是一个单位 F_k 贡献的次数，令其位 A ，而 $F_k = x$ 即有 x 个单位就是 $Ax \equiv F_n \pmod{998244352}$ 。

使用 BSGS 求出 F_n 后求一个逆元即可。注意这里模数不是质数，要用扩欧求解。

扩展 BSGS 算法

解决的是 $a \not\perp p$ 的情况。

扩展 BSGS 算法

解决的是 $a \not\perp p$ 的情况。

写出式子 $a^x \equiv b \pmod{m}$, 为了能够让 a 与 m 互质, 我们设 $d_1 = \gcd(a, m)$, 于是 $\frac{a}{d_1} a^{x-1} \equiv \frac{b}{d_1} \pmod{\frac{m}{d_1}}$, 若此时 $b \not\perp d_1$ 那么就无解了。



扩展 BSGS 算法

解决的是 $a \not\perp p$ 的情况。

写出式子 $a^x \equiv b \pmod{m}$, 为了能够让 a 与 m 互质, 我们设 $d_1 = \gcd(a, m)$, 于是 $\frac{a}{d_1} a^{x-1} \equiv \frac{b}{d_1} \pmod{\frac{m}{d_1}}$, 若此时 $b \not\perp d_1$ 那么就无解了。

这个时候左边和右边肯能仍不互质, 于是我们继续除, 设 $d_2 = \gcd(a, \frac{m}{d_1})$, 于是

$$\frac{a^2}{d_1 d_2} a^{x-2} \equiv \frac{b}{d_1 d_2} \pmod{\frac{m}{d_1 d_2}}$$



扩展 BSGS 算法

解决的是 $a \not\perp p$ 的情况。

写出式子 $a^x \equiv b \pmod{m}$, 为了能够让 a 与 m 互质, 我们设 $d_1 = \gcd(a, m)$, 于是 $\frac{a}{d_1} a^{x-1} \equiv \frac{b}{d_1} \pmod{\frac{m}{d_1}}$, 若此时 $b \not\perp d_1$ 那么就无解了。

这个时候左边和右边肯能仍不互质, 于是我们继续除, 设 $d_2 = \gcd(a, \frac{m}{d_1})$, 于是

$$\frac{a^2}{d_1 d_2} a^{x-2} \equiv \frac{b}{d_1 d_2} \pmod{\frac{m}{d_1 d_2}}$$

最后经过 $\log$ 次变换后, 我们令 $D = d_1 d_2 \dots d_k$, 我们得到了 $\frac{a^k}{D} a^{x-k} \equiv \frac{b}{D} \pmod{\frac{m}{D}}$



扩展 BSGS 算法

解决的是 $a \not\perp p$ 的情况。

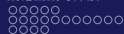
写出式子 $a^x \equiv b \pmod{m}$, 为了能够让 a 与 m 互质, 我们设 $d_1 = \gcd(a, m)$, 于是 $\frac{a}{d_1} a^{x-1} \equiv \frac{b}{d_1} \pmod{\frac{m}{d_1}}$, 若此时 $b \not\perp d_1$ 那么就无解了。

这个时候左边和右边肯能仍不互质, 于是我们继续除, 设 $d_2 = \gcd(a, \frac{m}{d_1})$, 于是

$$\frac{a^2}{d_1 d_2} a^{x-2} \equiv \frac{b}{d_1 d_2} \pmod{\frac{m}{d_1 d_2}}$$

最后经过 $\log$ 次变换后, 我们令 $D = d_1 d_2 \dots d_k$, 我们得到了 $\frac{a^k}{D} a^{x-k} \equiv \frac{b}{D} \pmod{\frac{m}{D}}$

因为此时 $a \perp \frac{m}{D}$ 且 $\frac{a^k}{D} \perp \frac{m}{D}$, 所以把 $\frac{a^k}{D}$ 除到右边后就是朴素 bsgs。



卢卡斯定理

定理

当满足 p 是质数时，有：

$$\binom{n}{m} \equiv \binom{n/p}{m/p} \binom{n \bmod p}{m \bmod p} \pmod{p}$$

卢卡斯定理

定理

当满足 p 是质数时，有：

$$\binom{n}{m} \equiv \binom{n/p}{m/p} \binom{n \bmod p}{m \bmod p} \pmod{p}$$

作用是求解 n, m 特别大，但是 p 相对较小的组合数。

卢卡斯定理

定理

当满足 p 是质数时，有：

$$\binom{n}{m} \equiv \binom{n/p}{m/p} \binom{n \bmod p}{m \bmod p} \pmod{p}$$

作用是求解 n, m 特别大，但是 p 相对较小的组合数。

推论

$$\binom{n}{m} \bmod 2 = [m \subseteq n]$$

例题洛谷 P2480 古代猪文

例题洛谷 P2480 古代猪文

可以发现模数是质数，要对质数取模，所以是要对模数 $-1 = 999911658$ 取模，这不是一个质数，没法直接卢卡斯定理。

例题洛谷 P2480 古代猪文

可以发现模数是质数，要对质数取模，所以是要对模数 $-1 = 999911658$ 取模，这不是一个质数，没法直接卢卡斯定理。

注意到 $999911658 = 2 \times 3 \times 4679 \times 35617$ ，所以对其每个质因子用扩展卢卡斯定理分别算一次，最后中国剩余定理合并就可以了。

扩展卢卡斯

现在模数不一定是质数。

扩展卢卡斯

现在模数不一定是质数。

可以学习上一页的题目，把模数分解成若干个质数的积，对每个分别做卢卡斯定理最后中国剩余定理合并。

扩展卢卡斯

现在模数不一定是质数。

可以学习上一页的题目，把模数分解成若干个质数的积，对每个分别做卢卡斯定理最后中国剩余定理合并。

问题在于分解出的不一定是一个单纯的质数 p ，可能是 p^α ，所以我们要对模数形如 p^α 的问题求解。



扩展卢卡斯

现在模数不一定是质数。

可以学习上一页的题目，把模数分解成若干个质数的积，对每个分别做卢卡斯定理最后中国剩余定理合并。

问题在于分解出的不一定是一个单纯的质数 p ，可能是 p^α ，所以我们要对模数形如 p^α 的问题求解。

考虑组合数公式 $\binom{n}{m} \equiv \frac{n!}{m!(n-m)!} \pmod{p^\alpha}$ ，我们为了求逆要保证分子分母中不含 p 的倍数，于是考虑把 p 的倍数提出来。

扩展卢卡斯

现在模数不一定是质数。

可以学习上一页的题目，把模数分解成若干个质数的积，对每个分别做卢卡斯定理最后中国剩余定理合并。

问题在于分解出的不一定是一个单纯的质数 p ，可能是 p^α ，所以我们要对模数形如 p^α 的问题求解。

考虑组合数公式 $\binom{n}{m} \equiv \frac{n!}{m!(n-m)!} \pmod{p^\alpha}$ ，我们为了求逆要保证分子分母中不含 p 的倍数，于是考虑把 p 的倍数提出来。

设 $v(n)$ 表示 $n!$ 中 p 的幂次，显然 $v(n) = \sum_i \lfloor n/p^i \rfloor$ ，再设 $r(n) = \frac{n!}{v(n)}$ ，那么

$$\binom{n}{m} \equiv \frac{r(n)}{r(m)r(n-m)} p^{v(n)-v(m)-v(n-m)} \pmod{p^\alpha}$$

扩展卢卡斯

考虑如何求 $r(n)$ ，考虑把 $n!$ 中 p 的倍数先扔到一边，得到

$$[1 \times 2 \times \cdots \times (p-1)] \times p \times [p+1 \times p+2 \times \cdots \times (2p-1)] \times 2p \times \cdots$$



扩展卢卡斯

考虑如何求 $r(n)$ ，考虑把 $n!$ 中 p 的倍数先扔到一边，得到

$$[1 \times 2 \times \cdots \times (p-1)] \times p \times [p+1 \times p+2 \times \cdots \times (2p-1)] \times 2p \times \cdots$$

对于大于 p^α 的部分是循环的，于是我们可以花 $O(p^\alpha)$ 的复杂度预处理每个中括号内部的值，最后直接 $O(1)$ 查询。



扩展卢卡斯

考虑如何求 $r(n)$, 考虑把 $n!$ 中 p 的倍数先扔到一边, 得到

$$[1 \times 2 \times \cdots \times (p-1)] \times p \times [p+1 \times p+2 \times \cdots \times (2p-1)] \times 2p \times \cdots$$

对于大于 p^α 的部分是循环的, 于是我们可以花 $O(p^\alpha)$ 的复杂度预处理每个中括号内部的值, 最后直接 $O(1)$ 查询。

对于中括号外 p 的倍数, 把他们统一除 p 后就是 $1 \times 2 \times \cdots \lfloor n/p \rfloor$ 即 $\lfloor n/p \rfloor!$, 转化为求解 $r(\lfloor n/p \rfloor)$



扩展卢卡斯

考虑如何求 $r(n)$ ，考虑把 $n!$ 中 p 的倍数先扔到一边，得到

$$[1 \times 2 \times \cdots \times (p-1)] \times p \times [p+1 \times p+2 \times \cdots \times (2p-1)] \times 2p \times \cdots$$

对于大于 p^α 的部分是循环的，于是我们可以花 $O(p^\alpha)$ 的复杂度预处理每个中括号内部的值，最后直接 $O(1)$ 查询。

对于中括号外 p 的倍数，把他们统一除 p 后就是 $1 \times 2 \times \cdots \lfloor n/p \rfloor$ 即 $\lfloor n/p \rfloor!$ ，转化为求解 $r(\lfloor n/p \rfloor)$

时间复杂度 $O(\sum p^\alpha + \log^2 p)$ 。

扩展欧拉定理

先前给出的欧拉定理中有 $a \perp m$ 的限制，现在我们讨论没有限制的欧拉定理。

扩展欧拉定理

先前给出的欧拉定理中有 $a \perp m$ 的限制，现在我们讨论没有限制的欧拉定理。

定理

$$a^n \equiv \begin{cases} a^{n \bmod \varphi(m)} & (a \perp m) \\ a^n & (a \not\perp m, n < \varphi(m)) \\ a^{(n \bmod \varphi(m)) + \varphi(m)} & (a \not\perp m, n \geq \varphi(m)) \end{cases} \pmod{m}$$

例题洛谷 P4139 上帝与集合的正确用法

例题洛谷 P4139 上帝与集合的正确用法

令 $f_m = 2^{2^{2^{\dots}}} \bmod m$, 那个根据扩展欧拉定理有 $f_m = 2^{f_{\varphi(p)} + \varphi(p)}$, 边界是 $f_1 = 0$

例题洛谷 P4139 上帝与集合的正确用法

令 $f_m = 2^{2^{2^{\dots}}} \bmod m$, 那个根据扩展欧拉定理有 $f_m = 2^{f_{\varphi(p)} + \varphi(p)}$, 边界是 $f_1 = 0$
这里有一个结论, 是 x 不断递归取 $\varphi(x)$, 那么取 $O(\log x)$ 次就能取到 1。

例题洛谷 P4139 上帝与集合的正确用法

令 $f_m = 2^{2^{2^{\dots}}} \bmod m$, 那个根据扩展欧拉定理有 $f_m = 2^{f_{\varphi(p)} + \varphi(p)}$, 边界是 $f_1 = 0$

这里有一个结论, 是 x 不断递归取 $\varphi(x)$, 那么取 $O(\log x)$ 次就能取到 1。

证明考虑观察欧拉函数的定义式, 可以发现若 x 无质因子 2 那么 $\varphi(x)$ 一定存在, 若存在那么 $\varphi(x)$ 一定减半。

Miller-Rabin 素性测试

快速检测一个数 n 是否为质数。

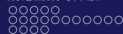
Miller-Rabin 素性测试

快速检测一个数 n 是否为质数。

利用了费马小定理和二次判断定理，即

定理

若 n 是奇质数，那么 $x^2 \equiv 1 \pmod{n}$ 的解要么是 $x = 1$ ，要么是 $x = n - 1$ 。



Miller-Rabin 素性测试

快速检测一个数 n 是否为质数。

利用了费马小定理和二次判断定理，即

定理

若 n 是奇质数，那么 $x^2 \equiv 1 \pmod{n}$ 的解要么是 $x=1$ ，要么是 $x=n-1$ 。

素性测试的基本思想是，拿任意一个数 x 去检查上面的性质，如果不成立就 hack 掉。

Miller-Rabin 素性测试

快速检测一个数 n 是否为质数。

利用了费马小定理和二次判断定理，即

定理

若 n 是奇质数，那么 $x^2 \equiv 1 \pmod{n}$ 的解要么是 $x=1$ ，要么是 $x=n-1$ 。

素性测试的基本思想是，拿任意一个数 x 去检查上面的性质，如果不成立就 hack 掉。

我们先用费马小定理，如果 $x^{n-1} \not\equiv 1 \pmod{n}$ 那么就直接 hack 掉，利用群论可以证明至少一半的数可以被这个方式 hack 掉。



Miller-Rabin 素性测试

快速检测一个数 n 是否为质数。

利用了费马小定理和二次判断定理，即

定理

若 n 是奇质数，那么 $x^2 \equiv 1 \pmod{n}$ 的解要么是 $x=1$ ，要么是 $x=n-1$ 。

素性测试的基本思想是，拿任意一个数 x 去检查上面的性质，如果不成立就 hack 掉。

我们先用费马小定理，如果 $x^{n-1} \not\equiv 1 \pmod{n}$ 那么就直接 hack 掉，利用群论可以证明至少一半的数可以被这个方式 hack 掉。

接下来我们用二次判断定理，令 $n-1 = a \times 2^b$ ，然后我们先算出 x^a ，然后不断平方来进行二次判断定理的检测。这部检测也有很多数无法测出。



Miller-Rabin 素性测试

快速检测一个数 n 是否为质数。

利用了费马小定理和二次判断定理，即

定理

若 n 是奇质数，那么 $x^2 \equiv 1 \pmod{n}$ 的解要么是 $x=1$ ，要么是 $x=n-1$ 。

素性测试的基本思想是，拿任意一个数 x 去检查上面的性质，如果不成立就 hack 掉。

我们先用费马小定理，如果 $x^{n-1} \not\equiv 1 \pmod{n}$ 那么就直接 hack 掉，利用群论可以证明至少一半的数可以被这个方式 hack 掉。

接下来我们用二次判断定理，令 $n-1 = a \times 2^b$ ，然后我们先算出 x^a ，然后不断平方来进行二次判断定理的检测。这部检测也有很多数无法测出。

虽然单一测试很难测出所有合数，但是一旦两者同时进行就能测出大部分合数。为了使得测试准确度更高我们可以选择多个底数 x 。已经通过枚举证明选择前 12 个质数为底数可以完全正确地检测出 $[1, 2^{64})$ 中的所有合数。

二次剩余

用于求解满足 m 是奇质数条件下形如 $x^2 \equiv a \pmod{m}$ 的方程（就是模意义下开根），我们称 a 是模 m 意义下的二次剩余。

二次剩余

用于求解满足 m 是奇质数条件下形如 $x^2 \equiv a \pmod{m}$ 的方程（就是模意义下开根），我们称 a 是模 m 意义下的二次剩余。

首先给出判断 a 为是否二次剩余的方法：欧拉判别法

结论

若 $a^{\frac{p-1}{2}} \equiv 1 \pmod{m}$ 那么有解，若 $a^{\frac{p-1}{2}} \equiv -1 \pmod{m}$ 那么无解。

二次剩余

用于求解满足 m 是奇质数条件下形如 $x^2 \equiv a \pmod{m}$ 的方程（就是模意义下开根），我们称 a 是模 m 意义下的二次剩余。

首先给出判断 a 为是否二次剩余的方法：欧拉判别法

结论

若 $a^{\frac{p-1}{2}} \equiv 1 \pmod{m}$ 那么有解，若 $a^{\frac{p-1}{2}} \equiv -1 \pmod{m}$ 那么无解。

证明考虑先证 $a^{\frac{p-1}{2}} \equiv \pm 1 \pmod{m}$ ，然后证 $a^{\frac{p-1}{2}} \equiv -1 \pmod{m}$ 是无解的充要条件即可。

二次剩余

用于求解满足 m 是奇质数条件下形如 $x^2 \equiv a \pmod{m}$ 的方程（就是模意义下开根），我们称 a 是模 m 意义下的二次剩余。

首先给出判断 a 为是否二次剩余的方法：欧拉判别法

结论

若 $a^{\frac{p-1}{2}} \equiv 1 \pmod{m}$ 那么有解，若 $a^{\frac{p-1}{2}} \equiv -1 \pmod{m}$ 那么无解。

证明考虑先证 $a^{\frac{p-1}{2}} \equiv \pm 1 \pmod{m}$ ，然后证 $a^{\frac{p-1}{2}} \equiv -1 \pmod{m}$ 是无解的充要条件即可。

同样由于 $a^2 \equiv (m-a)^2$ ，故可以感性理解为一个数 x 有且仅有另一个与他平方相同的数，故模 p 下的二次剩余恰有 $\frac{p-1}{2}$ 个。

二次剩余

我们使用 Cipolla 算法来求解二次剩余 a 。

二次剩余

我们使用 Cipolla 算法来求解二次剩余 a 。

首先我们随便找一个 b 使得 $b^2 - a$ 不是二次剩余，随机化即可。

二次剩余

我们使用 Cipolla 算法来求解二次剩余 a 。

首先我们随便找一个 b 使得 $b^2 - a$ 不是二次剩余，随机化即可。

随后设 $w^2 = b^2 - a$ ，因为不存在二次剩余所以同余系中不存在 w ，故可以理解为一种虚数，只是为了满足上述条件。

二次剩余

我们使用 Cipolla 算法来求解二次剩余 a 。

首先我们随便找一个 b 使得 $b^2 - a$ 不是二次剩余，随机化即可。

随后设 $w^2 = b^2 - a$ ，因为不存在二次剩余所以同余系中不存在 w ，故可以理解为一种虚数，只是为了满足上述条件。

随后我们扩域，把每个数扩展为 $A + Bw$ 的形式，容易发现扩域后仍然是环。

二次剩余

我们使用 Cipolla 算法来求解二次剩余 a 。

首先我们随便找一个 b 使得 $b^2 - a$ 不是二次剩余，随机化即可。

随后设 $w^2 = b^2 - a$ ，因为不存在二次剩余所以同余系中不存在 w ，故可以理解作为一种虚数，只是为了满足上述条件。

随后我们扩域，把每个数扩展为 $A + Bw$ 的形式，容易发现扩域后仍然是环。

于是我们有

$$c = a^2 - a^2 + c = a^2 - w^2 = (a + w)(a - w) = (a^p + w^p)(a + w) = (a + w)^{p+1},$$

中间利用了 $(a + b)^p = a^p + b^p$ 和 $w^p = -w$ 。

二次剩余

我们使用 Cipolla 算法来求解二次剩余 a 。

首先我们随便找一个 b 使得 $b^2 - a$ 不是二次剩余，随机化即可。

随后设 $w^2 = b^2 - a$ ，因为不存在二次剩余所以同余系中不存在 w ，故可以理解为一种虚数，只是为了满足上述条件。

随后我们扩域，把每个数扩展为 $A + Bw$ 的形式，容易发现扩域后仍然是环。

于是我们有

$$c = a^2 - a^2 + c = a^2 - w^2 = (a + w)(a - w) = (a^p + w^p)(a + w) = (a + w)^{p+1},$$

中间利用了 $(a + b)^p = a^p + b^p$ 和 $w^p = -w$ 。

于是 $x = (a + w)^{\frac{p+1}{2}}$ ，扩域后快速幂即可。

没有二次剩余？

如果题目中没有二次剩余，但是需要用，怎么办？

没有二次剩余？

如果题目中没有二次剩余，但是需要用，怎么办？

上面已经给出了一个方法，扩域，把所有数变成 $A + B\sqrt{C}$ 的形式即可。

例题 WC2021 斐波那契

例题 WC2021 斐波那契

我们将系数分离容易发现 $F_n = af_{n-1} + bf_n$, f_n 表示斐波那契数列的第 n 项。



例题 WC2021 斐波那契

我们将系数分离容易发现 $F_n = af_{n-1} + bf_n$, f_n 表示斐波那契数列的第 n 项。取 $b = -b$, 那么我们要求的是最小的 x 满足 $af_{x-1} \equiv bf_x \pmod{m}$, 如果 m 是质数那么直接移项即可独立两侧然后预处理。可惜并不是, 所以需要一些处理。



例题 WC2021 斐波那契

我们将系数分离容易发现 $F_n = af_{n-1} + bf_n$, f_n 表示斐波那契数列的第 n 项。取 $b = -b$, 那么我们要求的是最小的 x 满足 $af_{x-1} \equiv bf_x \pmod{m}$, 如果 m 是质数那么直接移项即可独立两侧然后预处理。可惜并不是, 所以需要一些处理。接下来的推到过程需要两个关键的性质

- ▶ $A \equiv B \pmod{m}$ 等价于 $\frac{A}{d} \equiv \frac{B}{d} \pmod{\frac{m}{d}}$, 其中 $d \mid \gcd(A, B, m)$ 。
- ▶ 若有 $A \equiv B \pmod{m}$, $d \mid \gcd(A, m)$, 那么 $d \mid B$ 。



例题 WC2021 斐波那契

我们将系数分离容易发现 $F_n = af_{n-1} + bf_n$, f_n 表示斐波那契数列的第 n 项。取 $b = -a$, 那么我们要求的是最小的 x 满足 $af_{x-1} \equiv bf_x \pmod{m}$, 如果 m 是质数那么直接移项即可独立两侧然后预处理。可惜并不是, 所以需要一些处理。接下来的推到过程需要两个关键的性质

- ▶ $A \equiv B \pmod{m}$ 等价于 $\frac{A}{d} \equiv \frac{B}{d} \pmod{\frac{m}{d}}$, 其中 $d \mid \gcd(A, B, m)$ 。
- ▶ 若有 $A \equiv B \pmod{m}$, $d \mid \gcd(A, m)$, 那么 $d \mid B$ 。

首先我们取 $d = \gcd(a, b, m)$, 那么有

$$\frac{a}{d}f_{x-1} \equiv \frac{b}{d}f_x \pmod{\frac{m}{d}}$$



例题 WC2021 斐波那契

我们将系数分离容易发现 $F_n = af_{n-1} + bf_n$, f_n 表示斐波那契数列的第 n 项。取 $b = -a$, 那么我们要求的是最小的 x 满足 $af_{x-1} \equiv bf_x \pmod{m}$, 如果 m 是质数那么直接移项即可独立两侧然后预处理。可惜并不是, 所以需要一些处理。接下来的推到过程需要两个关键的性质

- ▶ $A \equiv B \pmod{m}$ 等价于 $\frac{A}{d} \equiv \frac{B}{d} \pmod{\frac{m}{d}}$, 其中 $d \mid \gcd(A, B, m)$ 。
- ▶ 若有 $A \equiv B \pmod{m}$, $d \mid \gcd(A, m)$, 那么 $d \mid B$ 。

首先我们取 $d = \gcd(a, b, m)$, 那么有

$$\frac{a}{d}f_{x-1} \equiv \frac{b}{d}f_x \pmod{\frac{m}{d}}$$

现在我们想让两边系数独立, 即 $\frac{b}{d}$ 和 f_{x-1} 分别除到对面。先考虑 $\frac{b}{d}$, 取 $d' = \gcd(\frac{b}{d}, \frac{m}{d})$, 因为 $d' \mid$ 左边, $d' \perp \frac{a}{d}$ 于是 $d' \mid f_{x-1}$



例题 WC2021 斐波那契

我们将系数分离容易发现 $F_n = af_{n-1} + bf_n$, f_n 表示斐波那契数列的第 n 项。取 $b = -b$, 那么我们要求的是最小的 x 满足 $af_{x-1} \equiv bf_x \pmod{m}$, 如果 m 是质数那么直接移项即可独立两侧然后预处理。可惜并不是, 所以需要一些处理。接下来的推到过程需要两个关键的性质

- ▶ $A \equiv B \pmod{m}$ 等价于 $\frac{A}{d} \equiv \frac{B}{d} \pmod{\frac{m}{d}}$, 其中 $d \mid \gcd(A, B, m)$ 。
- ▶ 若有 $A \equiv B \pmod{m}$, $d \mid \gcd(A, m)$, 那么 $d \mid B$ 。

首先我们取 $d = \gcd(a, b, m)$, 那么有

$$\frac{a}{d}f_{x-1} \equiv \frac{b}{d}f_x \pmod{\frac{m}{d}}$$

现在我们想让两边系数独立, 即 $\frac{b}{d}$ 和 f_{x-1} 分别除到对面。先考虑 $\frac{b}{d}$, 取 $d' = \gcd(\frac{b}{d}, \frac{m}{d})$, 因为 $d' \mid$ 左边, $d' \perp \frac{a}{d}$ 于是 $d' \mid f_{x-1}$

$$\frac{a}{d} \frac{f_{x-1}}{d'} \equiv \frac{b}{dd'} f_x \pmod{\frac{m}{dd'}}$$

这时 $\frac{b}{dd'} \perp \frac{m}{dd'}$, 可以直接除过去。

例题 WC2021 斐波那契

得到

$$\frac{a}{d} \left(\frac{b}{dd'} \right)^{-1} \frac{f_{x-1}}{d'} \equiv f_x \pmod{\frac{m}{dd'}}$$

现在我们把 $\frac{f_{x-1}}{d'}$ 除到对面。

例题 WC2021 斐波那契

得到

$$\frac{a}{d} \left(\frac{b}{dd'} \right)^{-1} \frac{f_{x-1}}{d'} \equiv f_x \pmod{\frac{m}{dd'}}$$

现在我们把 $\frac{f_{x-1}}{d'}$ 除到对面。

考虑故技重施，取 $d'' = \gcd(\frac{f_{x-1}}{d'}, \frac{m}{dd'})$ ，由于 $f_{x-1} \perp f_x$ ，所以 $d'' = 1$ ，可以直接除过去。

例题 WC2021 斐波那契

得到

$$\frac{a}{d} \left(\frac{b}{dd'} \right)^{-1} \frac{f_{x-1}}{d'} \equiv f_x \pmod{\frac{m}{dd'}}$$

现在我们把 $\frac{f_{x-1}}{d'}$ 除到对面。

考虑故技重施，取 $d'' = \gcd(\frac{f_{x-1}}{d'}, \frac{m}{dd'})$ ，由于 $f_{x-1} \perp f_x$ ，所以 $d'' = 1$ ，可以直接除过去。

上面的推到我们需要关注的是 d, d', x 的值，于是我们需要预处理所有形如

$\left(d, d', \frac{f_x}{\frac{f_{x-1}}{d}} \pmod{\frac{m}{dd'}} \right)$ 的三元组。直接枚举复杂度肯定爆炸。



例题 WC2021 斐波那契

得到

$$\frac{a}{d} \left(\frac{b}{dd'} \right)^{-1} \frac{f_{x-1}}{d'} \equiv f_x \pmod{\frac{m}{dd'}}$$

现在我们把 $\frac{f_{x-1}}{d'}$ 除到对面。

考虑故技重施，取 $d'' = \gcd(\frac{f_{x-1}}{d'}, \frac{m}{dd'})$ ，由于 $f_{x-1} \perp f_x$ ，所以 $d'' = 1$ ，可以直接除过去。

上面的推到我们需要关注的是 d, d', x 的值，于是我们需要预处理所有形如

$\left(d, d', \frac{f_x}{\frac{f_{x-1}}{d}} \pmod{\frac{m}{dd'}} \right)$ 的三元组。直接枚举复杂度肯定爆炸。

注意到 $d' | f_{x-1}$ 且 $\frac{f_{x-1}}{d'} \perp \frac{m}{dd'}$ ，故枚举 d, x 后 $d' = \gcd(f_{x-1}, \frac{m}{d})$ ，时间复杂度 $O((md(m) + n) \log m)$

积性函数

积性函数

函数 F 满足对于任意 $a, b (a \perp b)$ 都有 $F(a)F(b) = F(ab)$

完全积性函数

函数 F 满足对于任意 a, b 都有 $F(a)F(b) = F(ab)$



迪利克雷卷积

定义在数论函数上的乘法。

记 f 卷 g 为 $(f * g)(n) = \sum_{ab=n} f(a)g(b)$

迪利克雷卷积

定义在数论函数上的乘法。

记 f 卷 g 为 $(f * g)(n) = \sum_{ab=n} f(a)g(b)$

性质：两个积性函数卷起来还是积性函数。



常见数论函数

$$e(n) = [n = 1]$$

$$I(n) = 1$$

$$\mu(n) = \begin{cases} 0 & n = p^2 q \\ (-1)^k & n = p_1 p_2 \dots p_k \end{cases}$$

$$\varphi(n) = \sum_{i=1}^n [i \perp n]$$

$$d(n) = \sum_{i|n} 1$$

$$id(n) = n$$

$$\sigma(n) = \sum_{i|n} i$$

上述函数均为积性函数。

常见数论函数的关系

可以认为一个函数卷 I 就是做前缀和，卷 μ 就是做差分。

常见数论函数的关系

可以认为一个函数卷 I 就是做前缀和，卷 μ 就是做差分。
我们有

$$\mu \xrightarrow{*I} e \xrightarrow{*I} I \xrightarrow{*I} d$$

$$\varphi \xrightarrow{*I} id \xrightarrow{*I} \sigma$$



常见数论函数的关系

可以认为一个函数卷 I 就是做前缀和，卷 μ 就是做差分。
我们有

$$\mu \xrightarrow{*I} e \xrightarrow{*I} I \xrightarrow{*I} d$$

$$\varphi \xrightarrow{*I} id \xrightarrow{*I} \sigma$$

相反有

$$\mu \xleftarrow{* \mu} e \xleftarrow{* \mu} I \xleftarrow{* \mu} d$$

$$\varphi \xleftarrow{* \mu} id \xleftarrow{* \mu} \sigma$$

迪利克雷前缀和

就是把一个数 x 分解质因数 $p_1^{k_1} p_2^{k_2} \dots$ ，然后把每个质因数当成一位做高维前缀和。



迪利克雷前缀和

就是把一个数 x 分解质因数 $p_1^{k_1} p_2^{k_2} \dots$ ，然后把每个质因数当成一位做高维前缀和。

复杂度和埃氏筛是一样的 $O(n \log \log n)$ 。相当于卷 I 。但比朴素略快。

迪利克雷前缀和

就是把一个数 x 分解质因数 $p_1^{k_1} p_2^{k_2} \dots$ ，然后把每个质因数当成一位做高维前缀和。

复杂度和埃氏筛是一样的 $O(n \log \log n)$ 。相当于卷 I 。但比朴素略快。

既然有前缀和，那么就有差分。对其做高维差分后就相当于卷 μ 。

线性筛任意积性函数

线性筛任意积性函数

先线性筛出数组 g_x 表示 x 的最小的质因子 p^α ，因为线性筛保证了每个数都只会被最小质因子筛掉，所以是好维护的。

线性筛任意积性函数

先线性筛出数组 g_x 表示 x 的最小的质因子 p^α ，因为线性筛保证了每个数都只会被最小质因子筛掉，所以是好维护的。

最后要筛的积性函数 $f_x = f_{g_x} f_{x/g_x}$

整除分块

对于 $\sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor w_i$ 形式的求和，我们可以利用整除分块优化。

整除分块

对于 $\sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor w_i$ 形式的求和，我们可以利用整除分块优化。
 具体来说，注意到 $\left\lfloor \frac{n}{i} \right\rfloor$ 只有 $O(\sqrt{n})$ 种取值，所以我们枚举每种取值的区间然后算区间中 w 的区间和即可优化。



整除分块

对于 $\sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor w_i$ 形式的求和，我们可以利用整除分块优化。

具体来说，注意到 $\left\lfloor \frac{n}{i} \right\rfloor$ 只有 $O(\sqrt{n})$ 种取值，所以我们枚举每种取值的区间然后算区间中 w 的区间和即可优化。

那么怎么枚举区间呢，我们令左端点为 l ，那么最大的右端点就是 $r = \left\lfloor \frac{n}{\left\lfloor \frac{n}{l} \right\rfloor} \right\rfloor$



整除分块

对于 $\sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor w_i$ 形式的求和，我们可以利用整除分块优化。

具体来说，注意到 $\left\lfloor \frac{n}{i} \right\rfloor$ 只有 $O(\sqrt{n})$ 种取值，所以我们枚举每种取值的区间然后算区间中 w 的区间和即可优化。

那么怎么枚举区间呢，我们令左端点为 l ，那么最大的右端点就是 $r = \left\lfloor \frac{n}{\left\lfloor \frac{n}{l} \right\rfloor} \right\rfloor$

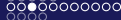
整除分块是筛法的基础，需要仔细掌握。

例题洛谷 P2260 [清华集训 2012] 模积和



例题洛谷 P2260 [清华集训 2012] 模积和

$$\begin{aligned}
 \text{ANS} &= \sum_{i=1}^n \sum_{j=1}^m (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{j} \rfloor \times j) [i \neq j] \\
 &= \sum_{i=1}^n \sum_{j=1}^m (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{j} \rfloor \times j) - \sum_{i=1}^{\min(n, m)} (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{i} \rfloor \times i)
 \end{aligned}$$



例题洛谷 P2260 [清华集训 2012] 模积和

$$\begin{aligned}
 \text{ANS} &= \sum_{i=1}^n \sum_{j=1}^m (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{j} \rfloor \times j) [i \neq j] \\
 &= \sum_{i=1}^n \sum_{j=1}^m (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{j} \rfloor \times j) - \sum_{i=1}^{\min(n, m)} (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{i} \rfloor \times i)
 \end{aligned}$$

对于前者，两个 $\sum$ 是相互独立的，可以分别用两次整除分块计算。



例题洛谷 P2260 [清华集训 2012] 模积和

$$\begin{aligned}
 \text{ANS} &= \sum_{i=1}^n \sum_{j=1}^m (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{j} \rfloor \times j) [i \neq j] \\
 &= \sum_{i=1}^n \sum_{j=1}^m (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{j} \rfloor \times j) - \sum_{i=1}^{\min(n, m)} (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{i} \rfloor \times i)
 \end{aligned}$$

对于前者，两个 $\sum$ 是相互独立的，可以分别用两次整除分块计算。

对于后者，拆开贡献后三项是好算的，唯一一项 $\sum \lfloor \frac{n}{i} \rfloor \lfloor \frac{m}{i} \rfloor$ 不是我们熟知的形式。



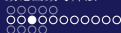
例题洛谷 P2260 [清华集训 2012] 模积和

$$\begin{aligned}
 \text{ANS} &= \sum_{i=1}^n \sum_{j=1}^m (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{j} \rfloor \times j) [i \neq j] \\
 &= \sum_{i=1}^n \sum_{j=1}^m (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{j} \rfloor \times j) - \sum_{i=1}^{\min(n, m)} (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{i} \rfloor \times i)
 \end{aligned}$$

对于前者，两个 $\sum$ 是相互独立的，可以分别用两次整除分块计算。

对于后者，拆开贡献后三项是好算的，唯一一项 $\sum \lfloor \frac{n}{i} \rfloor \lfloor \frac{m}{i} \rfloor$ 不是我们熟知的形式。

还是整除分块，因为 $\lfloor \frac{n}{i} \rfloor$ 和 $\lfloor \frac{m}{i} \rfloor$ ，各自分成了根号块，所以放在一起仍然是根号级别的，所以仍然整除分块就好了。



例题洛谷 P2260 [清华集训 2012] 模积和

$$\begin{aligned}
 \text{ANS} &= \sum_{i=1}^n \sum_{j=1}^m (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{j} \rfloor \times j) [i \neq j] \\
 &= \sum_{i=1}^n \sum_{j=1}^m (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{j} \rfloor \times j) - \sum_{i=1}^{\min(n, m)} (n - \lfloor \frac{n}{i} \rfloor \times i) (m - \lfloor \frac{m}{i} \rfloor \times i)
 \end{aligned}$$

对于前者，两个 $\sum$ 是相互独立的，可以分别用两次整除分块计算。

对于后者，拆开贡献后三项是好算的，唯一一项 $\sum \lfloor \frac{n}{i} \rfloor \lfloor \frac{m}{i} \rfloor$ 不是我们熟知的形式。

还是整除分块，因为 $\lfloor \frac{n}{i} \rfloor$ 和 $\lfloor \frac{m}{i} \rfloor$ ，各自分成了根号块，所以放在一起仍然是根号级别的，所以仍然整除分块就好了。

这题提供了我们多维整除分块的解决思路。

杜教筛

杜教筛的作用是快速求出某些特别积性函数的前缀和，有点是速度快，缺点是不通用。



杜教筛

杜教筛的作用是快速求出某些特别积性函数的前缀和，有点是速度快，缺点是不通用。

我们令 $S(n) = \sum_{i=1}^n f(i)$ ，于是对于任意数论函数 g ，有

$$\begin{aligned} \sum_{i=1}^n (f * g)(i) &= \sum_{i=1}^n \sum_{d|i} g(d) f\left(\frac{i}{d}\right) \\ &= \sum_{i=1}^n g(i) S\left(\left\lfloor \frac{n}{i} \right\rfloor\right) \end{aligned}$$



杜教筛

杜教筛的作用是快速求出某些特别积性函数的前缀和，有点是速度快，缺点是不通用。

我们令 $S(n) = \sum_{i=1}^n f(i)$ ，于是对于任意数论函数 g ，有

$$\begin{aligned}\sum_{i=1}^n (f * g)(i) &= \sum_{i=1}^n \sum_{d|i} g(d) f\left(\frac{i}{d}\right) \\ &= \sum_{i=1}^n g(i) S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)\end{aligned}$$

我们把我们需要的 $g(1)S(n)$ 单独拿出来得到：

$$\begin{aligned}g(1)S(n) &= \sum_{i=1}^n g(i) S\left(\left\lfloor \frac{n}{i} \right\rfloor\right) - \sum_{i=2}^n g(i) S\left(\left\lfloor \frac{n}{i} \right\rfloor\right) \\ &= \sum_{i=1}^n (f * g)(i) - \sum_{i=2}^n g(i) S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)\end{aligned}$$

杜教筛

所以我们只需要快速计算 $f * g$ 的前缀和，以及快速计算 g 的单点值，就能整除分块求解。

杜教筛

所以我们只需要快速计算 $f * g$ 的前缀和，以及快速计算 g 的单点值，就能整除分块求解。

但是这个整除分块是递归下去的，复杂度并不是简单的根号，注意到我们有

$$\left\lfloor \frac{a}{bc} \right\rfloor = \left\lfloor \frac{\left\lfloor \frac{a}{b} \right\rfloor}{c} \right\rfloor, \text{ 故只有约数位置有值。}$$

杜教筛

所以我们只需要快速计算 $f * g$ 的前缀和，以及快速计算 g 的单点值，就能整除分块求解。

但是这个整除分块是递归下去的，复杂度并不是简单的根号，注意到我们有

$$\left\lfloor \frac{a}{bc} \right\rfloor = \left\lfloor \frac{\left\lfloor \frac{a}{b} \right\rfloor}{c} \right\rfloor, \text{ 故只有约数位置有值。}$$

记忆化后复杂度就是 $O(\sum_{i=1}^{\sqrt{n}} \sqrt{\frac{n}{i}})$ ，积分得到 $O(n^{\frac{3}{4}})$ 。



杜教筛

所以我们只需要快速计算 $f * g$ 的前缀和，以及快速计算 g 的单点值，就能整除分块求解。

但是这个整除分块是递归下去的，复杂度并不是简单的根号，注意到我们有

$$\left\lfloor \frac{a}{bc} \right\rfloor = \left\lfloor \frac{\left\lfloor \frac{a}{b} \right\rfloor}{c} \right\rfloor, \text{ 故只有约数位置有值。}$$

记忆化后复杂度就是 $O(\sum_{i=1}^{\sqrt{n}} \sqrt{\frac{n}{i}})$ ，积分得到 $O(n^{\frac{3}{4}})$ 。

能不能更优呢？如果我们预处理出 $[1, B]$ 这个前缀的 S 的值，可以算出杜教筛复杂度变成 $O(\frac{n}{\sqrt{k}})$ ，加上预处理复杂度 $O(k)$ ，均摊一下取 $k = n^{2/3}$ 时复杂度最优，为 $O(n^{2/3})$

例题杜教筛求 μ

例题杜教筛求 μ

令 $f = \mu$, 取 $g = 1$, 那么 $f * g = e$, 满足上述求解条件。

例题杜教筛求 φ

例题杜教筛求 φ

令 $f = \varphi$, 取 $g = 1$, 那么 $f * g = id$, 满足上述求解条件。

Powerful Number

简称 PN ，也是用于筛特殊积性函数的前缀和，不过要求更严格，复杂度也更好。也可能与杜教筛配合使用。

Powerful Number

简称 PN ，也是用于筛特殊积性函数的前缀和，不过要求更严格，复杂度也更好。也可能与杜教筛配合使用。

定义

我们称 powerful number 为没有 1 次因子的数。

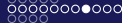
Powerful Number

简称 PN ，也是用于筛特殊积性函数的前缀和，不过要求更严格，复杂度也更好。也可能与杜教筛配合使用。

定义

我们称 powerful number 为没有 1 次因子的数。

显然 Powerful number 都能表示成 a^2b^3 。一个结论是 $\leq n$ 的 PN 数量为 $O(\sqrt{N})$ 。



Powerful Number

简称 PN ，也是用于筛特殊积性函数的前缀和，不过要求更严格，复杂度也更好。也可能与杜教筛配合使用。

定义

我们称 powerful number 为没有 1 次因子的数。

显然 Powerful number 都能表示成 a^2b^3 。一个结论是 $\leq n$ 的 PN 数量为 $O(\sqrt{N})$ 。

我们要筛积性函数 f ，于是我们构造满足 $g(p) = f(p)$ 的积性函数，以及 $f = g * h$ ，于是 h 也是积性的。



Powerful Number

简称 PN ，也是用于筛特殊积性函数的前缀和，不过要求更严格，复杂度也更好。也可能与杜教筛配合使用。

定义

我们称 powerful number 为没有 1 次因子的数。

显然 Powerful number 都能表示成 a^2b^3 。一个结论是 $\leq n$ 的 PN 数量为 $O(\sqrt{N})$ 。

我们要筛积性函数 f ，于是我们构造满足 $g(p) = f(p)$ 的积性函数，以及 $f = g * h$ ，于是 h 也是积性的。

注意到 $f(p) = g(1)h(p) + g(p)h(1)$ 且 $f(p) = g(p)$ ，于是 $h(p) = 0$ ，又由于 h 是积性的，故 h 只有在 PN 处有值。



Powerful Number

简称 PN ，也是用于筛特殊积性函数的前缀和，不过要求更严格，复杂度也更好。也可能与杜教筛配合使用。

定义

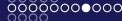
我们称 powerful number 为没有 1 次因子的数。

显然 Powerful number 都能表示成 a^2b^3 。一个结论是 $\leq n$ 的 PN 数量为 $O(\sqrt{N})$ 。

我们要筛积性函数 f ，于是我们构造满足 $g(p) = f(p)$ 的积性函数，以及 $f = g * h$ ，于是 h 也是积性的。

注意到 $f(p) = g(1)h(p) + g(p)h(1)$ 且 $f(p) = g(p)$ ，于是 $h(p) = 0$ ，又由于 h 是积性的，故 h 只有在 PN 处有值。

$$\sum_{i=1}^n f(i) = \sum_{i=1}^n \sum_{j|i} h(j) g\left(\frac{i}{j}\right) = \sum_{j=1}^n h(j) \sum_{i=1}^{n/j} g(i)$$



Powerful Number

简称 PN ，也是用于筛特殊积性函数的前缀和，不过要求更严格，复杂度也更好。也可能与杜教筛配合使用。

定义

我们称 powerful number 为没有 1 次因子的数。

显然 Powerful number 都能表示成 a^2b^3 。一个结论是 $\leq n$ 的 PN 数量为 $O(\sqrt{N})$ 。

我们要筛积性函数 f ，于是我们构造满足 $g(p) = f(p)$ 的积性函数，以及 $f = g * h$ ，于是 h 也是积性的。

注意到 $f(p) = g(1)h(p) + g(p)h(1)$ 且 $f(p) = g(p)$ ，于是 $h(p) = 0$ ，又由于 h 是积性的，故 h 只有在 PN 处有值。

$$\sum_{i=1}^n f(i) = \sum_{i=1}^n \sum_{j|i} h(j) g\left(\frac{i}{j}\right) = \sum_{j=1}^n h(j) \sum_{i=1}^{n/j} g(i)$$

只要满足 g 的前缀和是好求的即可 $O(\sqrt{n})$ 的复杂度内求解。

例题 HDU7186 Jo loves counting

定义 $Good_n$ 表示 n 的因子中与 n 质因数集合相等的数构成的集合。

定义函数 $f(x) = x \times |Good_x|$, 请你求出 $\sum_{i=1}^m f(i)$ 。

$$m \leq 10^{12}$$

例题 HDU7186 Jo loves counting

直接取 $g = id$, 满足上述要求, 直接套即可。

例题 loj 6053 简单的函数

例题 loj 6053 简单的函数

注意 f 在质数 p 处的取值, 除了 $p = 2$ 是 $p + 1$, 其余都是 $p - 1$ 。

例题 loj 6053 简单的函数

注意 f 在质数 p 处的取值，除了 $p = 2$ 是 $p + 1$ ，其余都是 $p - 1$ 。

考虑构造 $g(n) = \begin{cases} 3\varphi(n) & 2|n \\ \varphi(n) & \text{otherwise} \end{cases}$ ，那么显然 g 是积性的，且满足要求。现在变成要快速求 g 前缀和。



例题 loj 6053 简单的函数

注意 f 在质数 p 处的取值, 除了 $p = 2$ 是 $p + 1$, 其余都是 $p - 1$ 。

考虑构造 $g(n) = \begin{cases} 3\varphi(n) & 2|n \\ \varphi(n) & \text{otherwise} \end{cases}$, 那么显然 g 是积性的, 且满足要求。现

在变成要快速求 g 前缀和。

令 g 的前缀和 G , 那么有 $G(n) = \sum_{i=1}^n \varphi(i) + 2 \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \varphi(2i)$, 即要求一个 $\varphi(i)$ 的前缀和 S_1 , 以及 $\varphi(2i)$ 的前缀和 S_2 。



例题 loj 6053 简单的函数

注意 f 在质数 p 处的取值, 除了 $p = 2$ 是 $p + 1$, 其余都是 $p - 1$ 。

考虑构造 $g(n) = \begin{cases} 3\varphi(n) & 2|n \\ \varphi(n) & \text{otherwise} \end{cases}$, 那么显然 g 是积性的, 且满足要求。现

在变成要快速求 g 前缀和。

令 g 的前缀和 G , 那么有 $G(n) = \sum_{i=1}^n \varphi(i) + 2 \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \varphi(2i)$, 即要求一个 $\varphi(i)$ 的前缀和 S_1 , 以及 $\varphi(2i)$ 的前缀和 S_2 。

$$\begin{aligned} S_2(n) &= \sum_{i=1}^n \varphi(2i) \\ &= \sum_{i=1}^n [2 \wedge i] \varphi(i) + 2 \sum_{i=1}^n [2 \nmid i] \varphi(i) \\ &= S_1(n) + S_2(\lfloor n/2 \rfloor) \end{aligned}$$

莫比乌斯反演

利用的是 $\mu * 1 = e$ ，可以把 $[xxx]$ 的化成更容易计算的形式。

莫比乌斯反演

利用的是 $\mu * 1 = e$ ，可以把 $[xxx]$ 的化成更容易计算的形式。
举个例子，最常见的 $[gcd(i, j) = 1]$ 就可以变成 $\sum_{d|gcd(i, j)} \mu(d)$

例题 P2522 [HAOI2011] Problem b

例题 P2522 [HAOI2011] Problem b

首先可以二维差分转化为全部求前缀。



例题 P2522 [HAOI2011] Problem b

首先可以二维差分转化为全部求前缀。

开始推式子

$$\begin{aligned}
 \sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = k] &= \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{m}{k} \rfloor} [\gcd(i, j) = 1] \\
 &= \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{m}{k} \rfloor} \sum_{d \mid \gcd(i, j)} \mu(d) \\
 &= \sum_{d=1}^{\lfloor \frac{n}{k} \rfloor} \mu(d) \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} [d \mid i] \sum_{j=1}^{\lfloor \frac{m}{k} \rfloor} [d \mid j] \\
 &= \sum_{d=1}^{\min(\lfloor \frac{n}{k} \rfloor, \lfloor \frac{m}{k} \rfloor)} \mu(d) \lfloor \frac{n}{kd} \rfloor \lfloor \frac{m}{kd} \rfloor
 \end{aligned}$$



例题 P2522 [HAOI2011] Problem b

首先可以二维差分转化为全部求前缀。

开始推式子

$$\begin{aligned}
 \sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = k] &= \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{m}{k} \rfloor} [\gcd(i, j) = 1] \\
 &= \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{m}{k} \rfloor} \sum_{d \mid \gcd(i, j)} \mu(d) \\
 &= \sum_{d=1}^{\lfloor \frac{n}{k} \rfloor} \mu(d) \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} [d \mid i] \sum_{j=1}^{\lfloor \frac{m}{k} \rfloor} [d \mid j] \\
 &= \sum_{d=1}^{\min(\lfloor \frac{n}{k} \rfloor, \lfloor \frac{m}{k} \rfloor)} \mu(d) \lfloor \frac{n}{kd} \rfloor \lfloor \frac{m}{kd} \rfloor
 \end{aligned}$$

数论分块求解即可。

例题 P3704 [SDOI2017] 数字表格

例题 P3704 [SDOI2017] 数字表格

$$\prod_{d=1}^{\min(n,m)} \prod_{i=1}^n \prod_{j=1}^m [\gcd(i,j) = d] f_{\gcd(i,j)} = \prod_{d=1}^{\min(n,m)} f_d^{\sum_{i=1}^{n/d} \sum_{j=1}^{m/d} [\gcd(i,j)=1]}$$



例题 P3704 [SDOI2017] 数字表格

$$\prod_{d=1}^{\min(n,m)} \prod_{i=1}^n \prod_{j=1}^m [\gcd(i,j) = d] f_{\gcd(i,j)} = \prod_{d=1}^{\min(n,m)} f_d^{\sum_{i=1}^{n/d} \sum_{j=1}^{m/d} [\gcd(i,j)=1]}$$

看一下指数上的东西：

$$\sum_{i=1}^{n/d} \sum_{j=1}^{m/d} [\gcd(i,j) = 1] = \sum_{i=1}^{n/d} \mu(i) \left\lfloor \frac{n}{id} \right\rfloor \left\lfloor \frac{m}{id} \right\rfloor$$



例题 P3704 [SDOI2017] 数字表格

$$\prod_{d=1}^{\min(n,m)} \prod_{i=1}^n \prod_{j=1}^m [\gcd(i,j) = d] f_{\gcd(i,j)} = \prod_{d=1}^{\min(n,m)} f_d^{\sum_{i=1}^{n/d} \sum_{j=1}^{m/d} [\gcd(i,j)=1]}$$

看一下指数上的东西：

$$\sum_{i=1}^{n/d} \sum_{j=1}^{m/d} [\gcd(i,j) = 1] = \sum_{i=1}^{n/d} \mu(i) \left\lfloor \frac{n}{id} \right\rfloor \left\lfloor \frac{m}{id} \right\rfloor$$

推不动了，再扔回去，考虑枚举 id 再枚举 d ，令 $T = id$ ，有

$$\prod_{T=1}^{\min(n,m)} \prod_{d|T} f_d^{\mu(T/d) \lfloor n/T \rfloor \lfloor m/T \rfloor}$$

例题 P3704 [SDOI2017] 数字表格

把其中与 n, m 无关的单独拿出来，有

$$\prod_{T=1}^{\min(n,m)} \left(\prod_{d|T} f_d^{\mu(T/d)} \right)^{\lfloor n/T \rfloor \lfloor m/T \rfloor}$$



例题 P3704 [SDOI2017] 数字表格

把其中与 n, m 无关的单独拿出来，有

$$\prod_{T=1}^{\min(n,m)} \left(\prod_{d|T} f_d^{mu(T/d)} \right)^{\lfloor n/T \rfloor \lfloor m/T \rfloor}$$

中间的预处理，其余整除分块可以做到 $O(n + T\sqrt{n})$

谢谢大家。